1. Processes and threads

A process is a running program with its own address space, open files and resources. A thread is a unit of execution inside a process. The threads of one process share its address space and open files, but each thread has its own stack and its own registers. Creating a thread is cheaper than creating a process, and switching between two threads of the same process is faster than switching between two processes. Because threads share memory, they need synchronization whenever they touch shared data.

2. Mutual exclusion

A critical section is a piece of code that accesses shared data and must not be run by two threads at the same time. A mutex (mutual exclusion lock) guarantees that only one thread at a time can be inside the critical section: a thread locks the mutex before entering and unlocks it when leaving, and any other thread that tries to lock it blocks until it is released. A mutex has an owner, and only the thread that locked it should unlock it. With POSIX threads the calls are pthread_mutex_lock and pthread_mutex_unlock. Forgetting to unlock a mutex leaves every other thread waiting forever.

3. Semaphores and condition variables

A semaphore is an integer counter with two atomic operations. Wait, also called P, decrements the counter and blocks if it would become negative. Post, also called V, increments the counter and wakes a waiting thread. A semaphore initialised to 1 behaves like a mutex, but unlike a mutex it has no owner, so one thread can wait and another thread can post. A semaphore initialised to N lets up to N threads use a pool of N identical resources. A condition variable lets a thread sleep until another thread signals that a condition may have become true, and it is always used together with a mutex.

4. Deadlock

A deadlock is a situation in which a set of threads are all blocked, each one waiting for a resource held by another thread of the set. Four conditions must hold at the same time: mutual exclusion, hold and wait, no preemption, and circular wait. Mutual exclusion means a resource can be used by only one thread at a time. Hold and wait means a thread keeps the resources it already has while it waits for more. No preemption means a resource cannot be taken away from a thread and can only be released voluntarily. Circular wait means there is a cycle of threads in which each waits for a resource held by the next one. Deadlock can be prevented by breaking one of the four conditions. The most common method is to make all threads acquire locks in the same global order, which removes circular wait.

5. Paging

Paging splits virtual memory into fixed-size pages and physical memory into frames of the same size. A page table maps each virtual page to a frame. Because the page table lives in main memory, every access would cost an extra memory read. The TLB, or translation lookaside buffer, is a small and fast cache inside the MMU that stores recent translations. A TLB hit gives the frame number immediately, while a TLB miss forces a walk of the page table. When a page is not in memory, the hardware raises a page fault and the operating system loads the page from disk.

6. Page replacement

When memory is full, the operating system must evict a page. FIFO evicts the page that has been in memory the longest. LRU, least recently used, evicts the page that has not been used for the longest time and usually performs better, but it is costly to implement exactly. The optimal algorithm evicts the page that will not be needed for the longest time. It cannot be implemented, but it serves as a benchmark. FIFO can suffer from Belady's anomaly: giving a process more frames can increase its number of page faults. LRU does not suffer from this anomaly.